NFH-Kuza-Google Platform

Architecture Specification — Rural Service Delivery Marketplace

Domain Code: **local-services**

Version: 1.0

Date: April 2026

Status: Draft

1. Project Overview

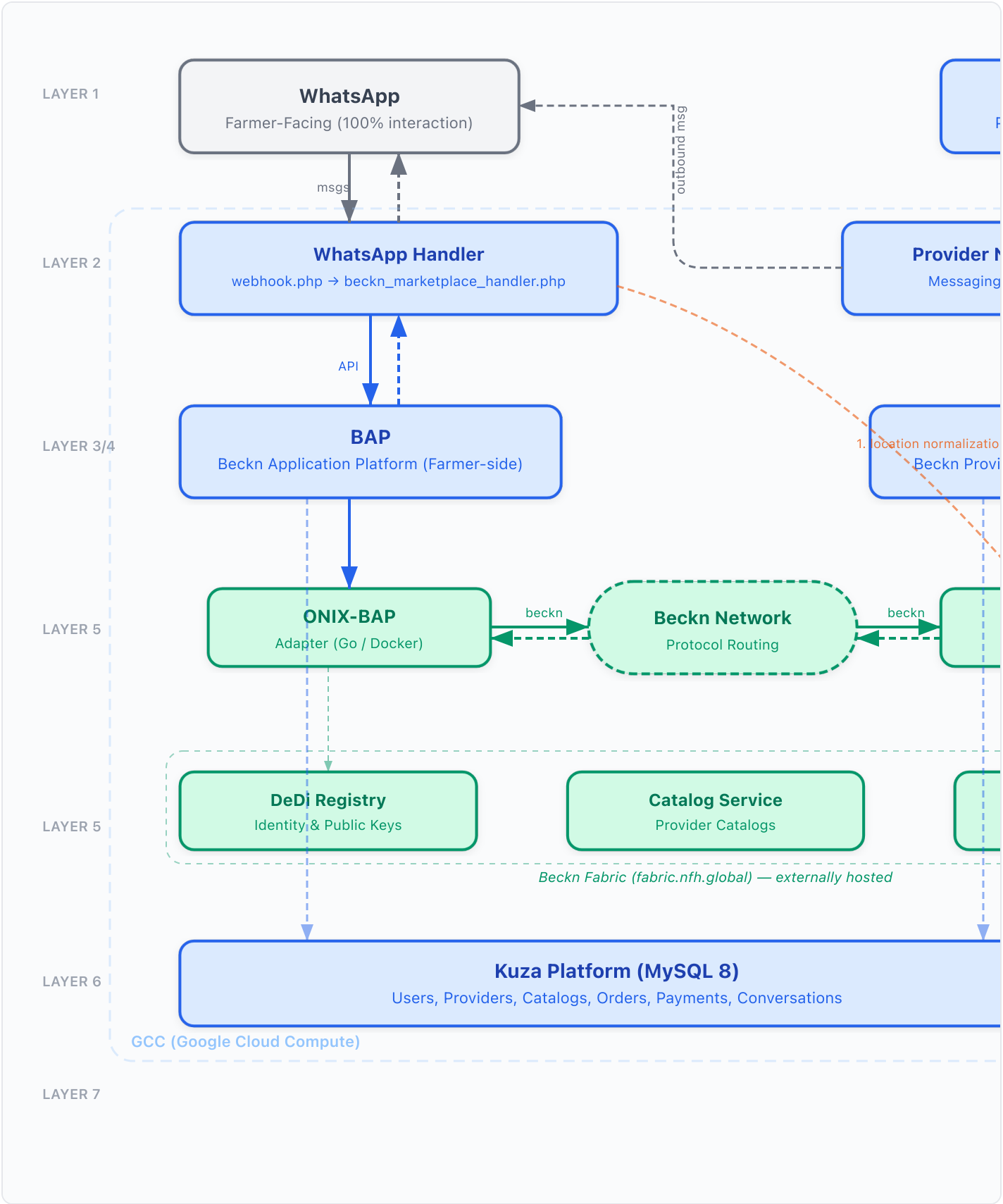
NFH-Kuza-Google is a rural service delivery marketplace that connects rural households with local service providers through WhatsApp and a lightweight web application. The platform is built on the **Beckn open network protocol**, enabling decentralized discovery and fulfillment of services across four domains: livelihood services, household services, market produce trading, and agricultural advisory.

> **Design Principle**
> 100% of farmer interaction occurs through WhatsApp — no app installation required. Providers use a lightweight Ionic Angular web app opened via WhatsApp button links.

Service Domains

- **Livelihood Services** — agricultural inputs, equipment rental, veterinary services, soil testing, etc.
- **Household Services** — plumbing, electrical, cleaning, construction, etc.
- **Market Produce** — connecting farmers with aggregators and buyers (reversed economics: buyer pays farmer)
- **Advisory** — agricultural content and guidance delivered as media through WhatsApp (no order lifecycle)

2. Architecture Diagram


Kuza-built components
Beckn infrastructure
Google services
WhatsApp / external
LAYER 1
WhatsApp
Farmer-Facing (100% interaction)
msgs
outbound msg
LAYER 2
WhatsApp Handler
webhook.php → beckn_marketplace_handler.php
Provider N
Messaging
API
LAYER 3/4
BAP
Beckn Application Platform (Farmer-side)
1. location normalizatio
Beckn Provi
LAYER 5
ONIX-BAP
Adapter (Go / Docker)
beckn
Beckn Network
Protocol Routing
beckn
LAYER 5
DeDi Registry
Identity & Public Keys
Catalog Service
Provider Catalogs
Beckn Fabric (fabric.nfh.global) — externally hosted
LAYER 6
Kuza Platform (MySQL 8)
Users, Providers, Catalogs, Orders, Payments, Conversations
GCC (Google Cloud Compute)
LAYER 7

3. Architecture Layers

3.1 Layer 1: Frontend Interfaces

WhatsApp (Farmer-Facing)

WhatsApp is the sole farmer-facing interface. 100% of farmer interactions — service discovery, provider selection, order confirmation, payment declaration, and feedback — occur through WhatsApp messages.

- **Interactive buttons** for small binary/ternary choices (Accept/Reject, Yes/No, payment method selection)
- **Numbered text lists** for longer option sets (provider lists, service categories, sub-categories)
- **Free text input** for location, quantity, and other open-ended fields
- Connected via **WhatsApp Cloud API**

Kuza Web App (Provider-Facing)

A lightweight Ionic Angular web application opened from WhatsApp button links — no native app installation required. Providers use it to:

- Manage their service catalog (add/edit services, set pricing)
- View and accept/reject incoming orders
- Update order status (in-progress, completed)
- Report payment received
- Manage their profile and service areas

3.2 Layer 2: WhatsApp Handler

> **Design Constraint**
>
> The existing `webhook.php` remains UNTOUCHED. Only routing logic is added to branch marketplace conversations to the new `beckn_marketplace_handler.php` based on trigger words or message IDs.

- **File:** `beckn_marketplace_handler.php` (new PHP file on Kuza Platform)

- **State machine:** manages conversational flow for the 4 marketplace domains (Livelihood, Household, Market, Advisory)
- **Translates** WhatsApp interactions into BAP API calls
- **Receives** async callbacks from BAP and formats WhatsApp response messages
- **Own tables:** conversation state, session tracking — separate from existing poultry chatbot tables (`chatbot_nodes` , `chatbot_transitions`)
- **Provider notifications** are triggered via a messaging queue/cron job, NOT through `webhook.php`

3.3 Layer 3: BAP (Beckn Application Platform)

The BAP is the farmer-side Beckn participant. It acts as the bridge between the WhatsApp Handler and the Beckn network.

- Receives translated requests from the WhatsApp Handler
- Constructs Beckn v2 protocol messages: `discover` , `select` , `init` , `confirm` , `status` , `cancel` , `rate`
- Sends protocol messages through the ONIX-BAP adapter
- Receives async callbacks (`on_discover` , `on_select` , `on_init` , `on_confirm` , `on_status` , `on_cancel` , `on_rate`)
- Pushes results back to WhatsApp Handler for farmer delivery
- Hosted by Kuza on GCC

3.4 Layer 4: BPP (Beckn Provider Platform)

The BPP is the provider-side Beckn participant. It manages the provider catalog, order lifecycle, and serves the provider web interface.

- Receives Beckn protocol requests from the network (via ONIX-BPP adapter)
- Connects to Kuza Platform to fetch provider catalogs, service areas, and pricing
- Manages order lifecycle on the provider side (create, accept, update status, complete)
- Sends async Beckn responses back through ONIX-BPP adapter
- Serves the **Kuza Web App** (provider interface) for catalog and order management
- Hosted by Kuza on GCC

3.5 Layer 5: Beckn Network Infrastructure

All Beckn network components are external to Kuza infrastructure, provided by Beckn Fabric.

ONIX Adapters

- Plugin-based Go servers running the `fidedocker/onix-adapter` Docker image
- Two instances: ONIX-BAP (farmer side) and ONIX-BPP (provider side)
- Handle message signing, signature validation, schema validation, and network routing
- Hosted on GCC as Docker containers

DeDi Registry

- Endpoint: `fabric.nfh.global/registry/dedi`
- Stores participant identity, public keys, and callback URIs
- Used for signature validation and dynamic routing between BAP and BPP

Catalog Service

- Endpoint: `fabric.nfh.global/beckn/catalog`
- BPP publishes provider catalogs here
- Queried by Discovery Service during `discover` requests

Discovery Service

- Handles `discover` requests from BAP
- Queries Catalog Service for matching providers
- Returns results to BAP via `on_discover` callback

3.6 Layer 6: Kuza Platform (Data Store)

MySQL 8 on GCC serves as the single source of truth. The Beckn network is stateless — all persistent data lives here.

Existing Tables Used

- `user` — agripreneurs (providers) and farmers
- Partner and organization data
- Content library (for advisory domain)

New Tables Required

Table	Purpose
`beckn_provider_catalog`	Services offered by providers, with pricing per service
`beckn_provider_service_areas`	County + Sub-County mappings for provider coverage areas
`beckn_subcategory_master`	Dynamic taxonomy of service sub-categories
`beckn_orders`	Order records with full lifecycle tracking
`beckn_order_status_history`	Audit trail of status transitions per order
`beckn_payments`	Payment records (farmer-side and provider-side)
`beckn_feedback`	Feedback and ratings from both parties
`beckn_conversation_state`	Marketplace conversation state machine data
`beckn_farmer_profile`	One-time farmer profile fields (location, preferences)

3.7 Layer 7: Google AI Services

Google AI is integrated directly into the request processing pipeline — not a standalone service. It is called from the PHP handler and the response flows back through the same handler to WhatsApp.

- **Location normalization (all flows):** Farmer enters free-text location (village/ward/county) → PHP handler sends text to Google AI → AI maps to structured County + Sub-County → PHP handler sends confirmation message to farmer on WhatsApp → farmer confirms or corrects.
- **Advisory content generation (Advisory flow):** Farmer selects Advisory → picks domain (Agri, Health, Finance) → category → sub-category → these attributes are sent from the PHP handler to Google AI → AI processes the request and generates the advisory response → PHP handler receives the AI-generated content → sends it back to farmer on WhatsApp. This flow goes through Beckn (BAP → BPP → Google AI → BPP → BAP) with Google AI called from the BPP side.
- Exposed as REST APIs on GCC
- Called synchronously from the PHP handler during request processing

- Co-located on GCC

4. Key Data Flows

Flow A: Farmer Discovers and Books a Service (Livelihood / Household)

```
Farmer (WhatsApp): Selects category --> service --> enters quantity
  --> webhook.php --> beckn_marketplace_handler.php
    Farmer types location (free text, e.g. "Naivasha")
    --> PHP Handler calls Google AI for location normalization
    <-- Google AI returns {county: "Nakuru", subCounty: "Naivasha"}
    --> WhatsApp: "Nakuru County, Naivasha Sub-County. Correct?" [Yes] [No]
    Farmer confirms [Yes]
    --> BAP Client --> ONIX-BAP --> [Beckn Network / Discovery Service] -->
ONIX-BPP --> BPP
      --> Kuza Platform (query providers by location + category + subCategory)
    <-- BPP --> ONIX-BPP --> [Beckn Network] --> ONIX-BAP --> BAP Client
  <-- beckn_marketplace_handler.php --> WhatsApp (list of providers with
prices)

Farmer selects provider --> select --> on_select (quote)
Farmer confirms        --> init   --> on_init  (payment terms)
Farmer books           --> confirm --> on_confirm (order created)
```

Flow B: Provider Receives and Accepts Order

```
Order confirmed in Beckn
  --> BPP --> Kuza Platform (create order record)
  --> Messaging Queue / Cron --> WhatsApp Cloud API --> Provider WhatsApp
    Provider taps [Accept] button in WhatsApp
      --> webhook.php --> provider routing --> BPP --> Beckn update
    Provider taps [View Details] link
      --> Opens Kuza Web App in browser --> reads from Kuza Platform
```

Flow C: Provider Completes Service, Farmer Confirms

```
Provider (Kuza Web App):
  --> Marks service complete, enters payment details, notes
  --> BPP --> Beckn status update --> ONIX-BAP --> BAP
  --> beckn_marketplace_handler.php --> WhatsApp to Farmer
    "Was the service completed?" [Yes] [No]
  Farmer taps [Yes]
  --> webhook.php --> beckn_marketplace_handler.php
    --> "How did you pay?" [Cash] [M-Pesa]
  Farmer selects payment method
  --> Order marked FARMER_CONFIRMED --> CLOSED
```

Flow D: Farmer Gets Advisory Content (Google AI Integrated)

```
Farmer (WhatsApp): Selects Advisory --> Domain (Agri/Health/Finance) -->
Category --> Sub-category
  --> webhook.php --> beckn_marketplace_handler.php
    --> BAP Client --> ONIX-BAP --> [Beckn Network] --> ONIX-BPP --> BPP
      --> BPP calls Google AI with {domain, category, subCategory,
farmerContext}
      <-- Google AI returns generated advisory content
    <-- BPP --> ONIX-BPP --> [Beckn Network] --> ONIX-BAP --> BAP Client
  <-- beckn_marketplace_handler.php --> WhatsApp (AI-generated advisory
message/content)

Google AI is called from BPP during on_discover processing.
Advisory is free, no order lifecycle. Content generated and delivered
immediately.
```

Flow E: Market Produce (Farmer Sells)

```
Same flow as A but with reversed economics:
- Farmer has produce to sell (quantity + type)
- discover returns aggregators/buyers with buying prices
- Aggregator collects produce from farmer
- Aggregator pays farmer (not farmer pays provider)
- Both parties confirm and provide feedback
```

5. Component Ownership

Component	Built By	Hosted On
WhatsApp Handler (`beckn_marketplace_handler.php`)	Kuza	GCC
BAP application logic	Kuza	GCC
BPP application logic	Kuza	GCC
Kuza Web App (provider interface)	Kuza	GCC
Provider WhatsApp notifications	Kuza	GCC
ONIX Adapters (BAP + BPP)	Beckn (open source, Go)	GCC (Docker)
DeDi Registry	Beckn	Beckn Fabric (external)
Catalog Service	Beckn	Beckn Fabric (external)
Discovery Service	Beckn	Beckn Fabric (external)
Google AI (location normalization + advisory content generation)	Google	GCC
MySQL database	Kuza	GCC

6. Database Schema

The following schema additions are required on MySQL 8 (GCC). These tables live alongside existing Kuza Platform tables and reference the existing `user` table via foreign keys.

Entity Relationship Summary

```
user (existing)
  |-- beckn_farmer_profile      (1:1, farmer-specific fields)
  |-- beckn_provider_catalog    (1:N, provider's services)
  |-- beckn_provider_service_areas (1:N, coverage areas)

beckn_subcategory_master        (standalone taxonomy)

beckn_orders
  |-- beckn_order_status_history (1:N, audit trail)
  |-- beckn_payments             (1:N, farmer + provider payments)
  |-- beckn_feedback             (1:N, ratings from both parties)

beckn_conversation_state        (keyed by phone + session)
```

Key Table Definitions

`beckn_orders`

```
order_id          BIGINT AUTO_INCREMENT PRIMARY KEY
order_uuid        VARCHAR(36) UNIQUE NOT NULL
beckn_order_id    VARCHAR(255)        -- Beckn network order ID
farmer_user_id    INT                 -- FK to user.id
provider_user_id  INT                 -- FK to user.id
category          VARCHAR(100)        -- livelihood | household | market | advisory
sub_category      VARCHAR(255)
service_name      VARCHAR(255)
quantity          DECIMAL(10,2)
unit              VARCHAR(50)
quoted_price      DECIMAL(10,2)
currency          VARCHAR(10) DEFAULT 'KES'
status            ENUM('ORDER_PLACED','PROVIDER_ACCEPTED','PROVIDER_REJECTED',
                       'IN_PROGRESS','COMPLETED','FARMER_CONFIRMED',
                       'CANCELLED','CLOSED')
farmer_location   VARCHAR(255)        -- normalized county/sub-county
provider_location VARCHAR(255)
payment_method    VARCHAR(50)         -- cash | mpesa
created_at        TIMESTAMP DEFAULT CURRENT_TIMESTAMP
updated_at        TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
partner_id        VARCHAR(50)         -- multi-tenancy scope
```

`beckn_conversation_state`

```
id                BIGINT AUTO_INCREMENT PRIMARY KEY
phone_number      VARCHAR(20) NOT NULL
session_id        VARCHAR(100)
current_flow      ENUM('livelihood','household','market','advisory')
current_step      VARCHAR(100)
state_data        JSON                -- accumulated selections, context
beckn_txn_id      VARCHAR(255)        -- Beckn transaction correlation
last_activity     TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
created_at        TIMESTAMP DEFAULT CURRENT_TIMESTAMP
partner_id        VARCHAR(50)
INDEX idx_phone_session (phone_number, session_id)
```

7. Deployment Topology

All Kuza-built components are deployed on Google Cloud Compute (GCC). Beckn Fabric services are externally hosted and accessed over HTTPS.

GCC Instance Layout

Component	Runtime	Port	Notes
Kuza Platform (PHP)	Apache + PHP 7.4	443	Existing platform + WhatsApp Handler + BAP + BPP
Kuza Web App	Nginx (static)	443	Ionic Angular build served as static files
ONIX-BAP	Docker (`fidedocker/onix-adapter`)	5001	BAP-side adapter configuration
ONIX-BPP	Docker (`fidedocker/onix-adapter`)	5002	BPP-side adapter configuration
Google AI APIs	Cloud Functions / API	443	Location normalization + Advisory content generation
MySQL 8	MySQL Server	3306	Single database instance

External Dependencies

Service	Endpoint	Protocol
WhatsApp Cloud API	`graph.facebook.com`	HTTPS REST
DeDi Registry	`fabric.nfh.global/registry/dedi`	HTTPS REST
Beckn Catalog Service	`fabric.nfh.global/beckn/catalog`	HTTPS REST
Beckn Discovery Service	`fabric.nfh.global`	HTTPS REST (Beckn v2)

Deployment Note

ONIX adapters run as Docker containers on the same GCC instance. They require outbound HTTPS access to Beckn Fabric endpoints and inbound access from the BAP/BPP application logic on localhost.